

CardiolA Acolhe

Assistente conversacional para acolhimento cardiológico educativo

FLASK	REACT/VITE	WATSON ASSISTANT	SEM IA GENERATIVA
-------	------------	------------------	-------------------

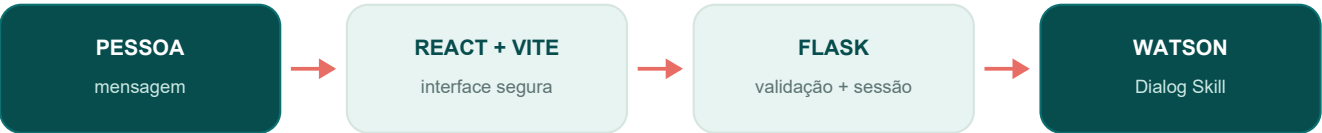
Objetivo

O CardiolA Acolhe organiza um relato fictício de sintomas antes de uma conversa com um profissional. Em poucos turnos, coleta sintoma, intensidade de 0 a 10 e duração, mantém essas informações no contexto e devolve um resumo estruturado. O protótipo é determinístico, acadêmico e não armazena conversas em banco de dados.

SEGURANÇA CLÍNICA

Não diagnostica, não prescreve e não substitui atendimento. Diante de dor torácica intensa, falta de ar importante, suor frio ou desmaio, orienta emergência ou SAMU 192.

Arquitetura



O React renderiza texto de forma segura e usa a API na mesma origem. O Flask valida entradas, oculta credenciais, normaliza a resposta e gerencia sessões stateful da API V2. A Dialog Skill concentra entendimento e decisões; nenhum modelo generativo participa do fluxo.

Camada	Responsabilidade	Evidência no repositório
Interface	Histórico, sugestões, Enter/botão, loading, erros, reset, alerta e painel NLP.	src/frontend
API	Health, chat e reset; limites; códigos 400/502/503; renovação única de sessão.	src/backend
NLU	7 intents, 3 entidades com sinônimos, @sys-number, contexto e fallback.	config/watson

Modelagem NLU e fluxo

Intents: #saudacao, #relatar_sintoma, #sinal_alerta, #preparar_consulta, #limites_assistente, #agradecimento e #despedida (7–8 exemplos cada). Entidades: @sintoma, @intensidade, @confirmacao e @sys-number. Contexto: \$sintoma, \$intensidade, \$duracao.

1 Emergência	2 Sintoma	3 Intensidade	4 Duração	5 Resumo	6 Apoio	7 Fallback
-----------------	--------------	------------------	--------------	-------------	------------	---------------

Implementação verificável

Contratos pequenos, respostas rastreáveis e limites explícitos

Contrato da API

Método	Rota	Contrato essencial
GET	/api/health	Estado do app e presença da configuração, sem revelar segredos.
POST	/api/chat	message + conversationId → reply, intent, confiança, entidades e urgent.
POST	/api/reset	Encerra/descarta a sessão e devolve conversationId nulo.

Entradas vazias ou acima de 500 caracteres retornam 400; configuração ausente, 503; falha do Watson, 502. Uma sessão 404/410 é recriada uma única vez. Mensagens clínicas e credenciais não são registradas.

Privacidade, acessibilidade e experiência

Privacidade por desenho	Sem banco; .env ignorado; health sem segredo; uso apenas de frases fictícias.
Interface acessível	aria-live, rótulos, foco por teclado, contraste, movimento reduzido e layout responsivo.
Resposta segura	Renderização de texto pelo React; urgência em destaque; fallback oferece caminhos de retomada.

Testes e resultados

Verificação	Resultado
Backend + export Watson	33/33 testes pytest aprovados em Python 3.14
Frontend	ESLint aprovado; build de produção Vite 8.3 aprovado
React → Flask (sem credencial)	Health, build servido e falha 503 segura conferidos no Chrome
React → Flask → Watson	Pendente: configuração acompanhada no plano Lite e smoke test real
Roteiro e vídeo	Roteiro de 2min32s pronto; gravação e link pendentes do usuário

Entregáveis e reprodutibilidade

O repositório contém export importável da Dialog Skill, API Flask testável por injeção, interface React/Vite, lockfile pnpm, CI, suíte automatizada, relatório-fonte e roteiro de 2min32s. As instruções do README reproduzem desenvolvimento, build integrado e smoke test. O repositório permanece privado por decisão do grupo, com o avaliador previamente convidado.

Grupo 7

Alice C. M. Assis — RM 566233 · Leonardo S. Souza — RM 563928 · Lucas B. Francelino — RM 561409 · Pedro L. T. Silva — RM 561644 · Vitor A. Bezerra — RM 563001
Coordenação: André Godoi Chiovato. Grupo com cinco integrantes; ponto adicional registrado separadamente.

Referências

IBM — Dialog Skills e APIs. Ministério da Saúde — [Infarto](#) e [SAMU 192](#). Código e documentação: README do projeto.